

Rapport de Travaux Pratiques

Monolithe Modulaire

Architecture Logicielle

Présenté par :

Haytham KENNOUZ

Encadré par :

Prof. ELBAGHAZAOUI

Année Universitaire 2025-2026

Table des matières

1	Introduction	2
2	Objectifs du TP	2
3	Analyse de l'Architecture	2
3.1	Modularity & Communication	2
4	Détails Techniques	3
4.1	Stack Technologique	3
4.2	Pattern DTO et Mapping	3
4.3	Logique métier : Création de commande	3
5	Conclusion	4

1 Introduction

L'architecture logicielle moderne évolue constamment pour répondre aux besoins de scalabilité et de maintenabilité. Le présent travail pratique porte sur la conception et l'implémentation d'un **Monolithe Modulaire**.

Contrairement à un monolithe traditionnel "spaghetti", le monolithe modulaire impose une séparation stricte des domaines métier au sein d'une seule unité de déploiement. Cette approche constitue une étape intermédiaire idéale avant une éventuelle transition vers des microservices, offrant un bon compromis entre simplicité opérationnelle et découplage logique.

2 Objectifs du TP

L'objectif principal était de transformer une application de commerce électronique monolithique en une structure modulaire en utilisant les technologies Java et Spring Boot. Les étapes clés incluaient :

- La mise en place d'une structure de packages orientée domaine.
- L'utilisation de **DTOs** (Data Transfer Objects) pour découpler les couches.
- L'automatisation du mapping avec **MapStruct**.
- La communication entre les modules (*Customer*, *Product*, *Order*) via des interfaces de services.

3 Analyse de l'Architecture

L'application est structurée autour de quatre modules principaux, chacun encapsulant ses propres entités, dépôts, services et contrôleurs :

Modules Implémentés

Customer : Gestion des comptes clients (Profil, authentification, vérification).

Product : Gestion du catalogue (Produits, catégories, prix).

Order : Gestion du cycle de vie des commandes, lié aux clients et aux produits.

Category : Organisation hiérarchique des produits.

3.1 Modularity & Communication

La communication entre les modules est réalisée par injection de dépendances. Par exemple, le module *Order* interagit avec *CustomerService* et *ProductService* pour valider les données lors de la création d'une commande.

4 Détails Techniques

4.1 Stack Technologique

- **Framework** : Spring Boot 3.2.2
- **Langage** : Java 17
- **Persistence** : Spring Data JPA + PostgreSQL
- **Productivité** : Lombok & MapStruct
- **Build** : Maven

4.2 Pattern DTO et Mapping

Pour assurer l'encapsulation, les entités JPA ne sont jamais exposées directement via l'API REST. Nous avons utilisé MapStruct pour générer automatiquement les convertisseurs.

Listing 1 – Exemple de Mapper MapStruct

```
1 @Mapper(componentModel = "spring")
2 public interface ProductMapper {
3     ProductDTO toProductDTO(Product product);
4     Product toProduct(CreateProductRequest request);
5 }
```

4.3 Logique métier : Création de commande

L'implémentation du service de commande illustre l'interaction entre modules :

Listing 2 – OrderServiceImpl.java (extrait)

```
1 @Transactional
2 public OrderDTO createOrder(CreateOrderRequest request) {
3     // Appel du module Customer
4     customerService.getCustomerById(request.getCustomerId());
5
6     // Calcul du prix total via le module Product
7     for (OrderItemRequest itemRequest : request.getItems()) {
8         ProductDTO product = productService.getProductById(
9             itemRequest.getProductId());
10        // ... logique de calcul
11    }
12    // ... enregistrement
13 }
```

5 Conclusion

Ce TP a permis de démontrer les avantages d’une architecture modulaire au sein d’un monolithe. Nous avons réussi à :

- Réduire le couplage entre les domaines métier.
- Améliorer la lisibilité du code grâce aux patterns DTO et Service.
- Faciliter l’évolution future vers une architecture distribuée.

En conclusion, le monolithe modulaire est une stratégie robuste pour les projets de taille moyenne, permettant une gestion saine de la complexité sans le surcoût opérationnel des microservices.